

RPi3B+ Debugging Investigation

Page Content

- [Tickets](#)
- [Current State](#)
- [Relevant Discussions](#)
- [Performed Investigation](#)
 - [Bare Metal Debugging](#)
 - [Using the telnet connection](#)
- [Conclusion](#)
- [Attachments](#)

Tickets

[SEOS-2275](#) - Getting issue details...

STATUS

Current State

As of now, runtime debugging of TRENTOS-M applications running on RPi3B+ is not working reliably. The reason for this is not currently known, but the behaviour is as follows.

After the board is powered on, it is possible to connect to it using OpenOCD with the configuration file provided in the SDK and this connection is successfully established without any warnings/errors.

After OpenOCD is running and the TCP port is exposed, it is possible to connect to it using any of the discussed interfaces - VS Code/**gdbgui**/command line gdb (they all use the same tool underneath - **gdb-multiarch**). This connection is also established without producing any errors and after the connection, the board is halted.

At this point the following functions are usable:

- Register inspection
- Memory inspection
- Overview of local variables
- Single stepping from the point where the program execution was halted - possible and does not cause the board to halt, but produces the following errors:

Error messages when single stepping

```
Error: cannot read system control register in this mode: (USR : 0x10)
Error: cannot read system control register in this mode: (USR : 0x10)
Error: cannot read system control register in this mode: (USR : 0x10)
Error: Opcode 0x0b10ec51, DSCR.ERR=1, DSCR.EL=1
Error: Opcode 0x0b11ec51, DSCR.ERR=1, DSCR.EL=1
[... message keeps repeating ...]
Error: Opcode 0x0b3fec51, DSCR.ERR=1, DSCR.EL=1
Error: Opcode 0x0a10eef1, DSCR.ERR=1, DSCR.EL=1
Error: Failed to read fpsr register
```

In case the breakpoint is set, after the program is released and it hits the set breakpoint, **the connection between OpenOCD and the board stops working** and OpenOCD produces the following output:

Error messages after hitting the breakpoint

```
Error: cannot read system control register in this mode: (USR : 0x10)
Polling target rpi3bplus.a53.0 failed, trying to reexamine
Info : rpi3bplus.a53.0: hardware has 6 breakpoints, 4 watchpoints
Error: aarch64_mmu: target rpi3bplus.a53.0 not halted
Error: aarch64_mmu: target rpi3bplus.a53.0 not halted
[... message keeps repeating ...]
Error: aarch64_mmu: target rpi3bplus.a53.0 not halted
Error: aarch64_mmu: target rpi3bplus.a53.0 not halted
Error: aarch64_mmu: target rpi3bplus.a53.0 not halted
Error: cannot read system control register in this mode: (USR : 0x10)
Polling target rpi3bplus.a53.0 failed, trying to reexamine
Info : rpi3bplus.a53.0: hardware has 6 breakpoints, 4 watchpoints
Error: cannot read system control register in this mode: (USR : 0x10)
Polling target rpi3bplus.a53.0 failed, trying to reexamine
Info : rpi3bplus.a53.0: hardware has 6 breakpoints, 4 watchpoints
```

Since the connection is lost at this point, naturally, it is not possible to resume the program execution nor to interact with the target in any way without a complete restart.

Relevant Discussions

- <https://sourceforge.net/p/openocd/mailman/openocd-user/?viewmonth=202004>
- <http://openocd.zylin.com/#c/5517/>
- <https://metebalci.com/blog/bare-metal-raspberry-pi-3b-jtag/>

Performed Investigation

Bare Metal Debugging

In addition to trying out debugging of TRENTOS-M systems, setting up and debugging a simple bare metal UART example which partially relies on the setup described here: <https://metebalci.com/blog/bare-metal-raspberry-pi-3b-jtag/> was tried out.

This setup indicates that the problem could be in the TRENTOS-M specificities because even though debugging does not work perfectly with this setup either (could be due to the ad hoc approach and that with more effort, it would work flawlessly) hitting the breakpoint in this setup does not cause errors. The behaviour is as follows:

- 32-bit programs:
 - the complete setup with a GUI is usable,
 - **setting breakpoints directly in code works,**
 - single-stepping, local variable inspection, register overview works,
 - **same error messages are produced as for single-stepping in the case of TRENTOS-M debugging (previous section) without noticeable impact on the experience (the connection between OpenOCD and the board is not lost)**
- 64-bit programs:
 - setting breakpoints does not work from the GUI nor from the **gdb** console if the line number is referenced the breakpoint is ignored
 - **setting breakpoints works when referencing the specific instruction address (e.g. b *0x80b8)**
 - single-stepping does not work "Cannot find bounds of current function" from **gdb**
 - **no error messages are produced when hitting the breakpoint**

The project used for this investigation is attached at the bottom of the page. Building the project is done by executing:

Building the 32-bit variant of the bare metal project

```
make all_32
```

for the 32-bit variant, or:

Building the 64-bit variant of the bare metal project

```
make all_64
```

for the 64-bit variant of the same demo. Running on the board is largely similar to running TRENTOS-M systems - it is necessary to first copy all the required files from the resources folder in the **sdk**, alongside **main_32.bin/main_64.bin** that result from the project build. Also, it is necessary to replace the kernel parameter in the **config.txt** file to match the name of the binary we are using. Finally, for the 64-bit variant, it is necessary to add the following line to the **config.txt**:

Modifying the config.txt file

```
arm_64bit=1
```

After the board is connected to the PC using the UART adapter and powered on, you should see the continuous output in the serial terminal (looping of all possible char values).

Using the telnet connection

In addition to using gdb as a "front-end" for the OpenOCD connection, it is possible to use the OpenOCD commands directly. The way to do this is to open a second terminal after the OpenOCD connection is established and type in the following command:

Starting the telnet connection to OpenOCD

```
telnet localhost 4444
```

At this point, the telnet terminal allows you to execute different OpenOCD commands described here: <http://www.openocd.org/doc/html/General-Commands.html>.

Some of the useful commands are:

- **targets** - displayed the list of available debug targets and their states (4 cores in case of RPi3B+)
- **targets <TARGET_NAME>** - switches the currently selected target
- **reg** - displays all the registers and their values
- **halt** - halts program execution
- **resume** - resumes program execution

Even though this approach is maybe not that useful for actual debugging, it is quite useful for investigating debug issues since it eliminates one or more intermediate tools (gdb, the GUI solutions) which allows us to narrow in on the problem.

Conclusion

The next step, in my opinion, would be to dive deeper into the OpenOCD manual and try to figure out what exactly happens once the target hits a breakpoint and if it is possible to solve the problem of losing the connection after hitting the breakpoint by modifying the OpenOCD configuration file. An interesting starting point would be to see if configuring the behaviour after different events described in section 11.5 here: <http://openocd.org/doc/html/CPU-Configuration.html> could make a difference since in the configuration file we already set the behaviour on some events (see lines 50 and 51 of https://bitbucket.hensoldt-cyber.systems/projects/SEOS/repos/sandbox/browse/resources/openocd_cfgs/rpi3.cfg). As for the error messages regarding the system control register and "Error: Opcode 0x0b10ec51, DSCR.ERR=1, DSCR.EL=1" which are printed when single-stepping without having any noticeable effect, I was not able to find any meaningful reference to this behaviour anywhere, and it could be that it can be ignored for the time being if we are able to fix the breakpoint issue which is the main blocker for the RPi3B+ runtime debugging to be fully usable at the moment. Also, since in the bare metal setup it is possible to use breakpoints, further attention to the possible cause in the TRENTOS-M setup should be investigated.

Attachments

File	Modified
ZIP Archive RPi3B_bare_metal.zip	Feb 18, 2021 by leobla01

Drag and drop to upload or [browse for files](#)